

Datové struktury a algoritmy

Přednáška 4, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

Dnešní témata

- Prezence (rezervováno/odevzdáno/přítomno): 4/4/4
 - Zásobník
 - Fronta
 - Dynamické programování
- **Výpočetní složitost** (opakování a dokončení)
 - Bonifikovaný kvíz 1: Výpočetní složitost složených funkcí
 - Rozklad na podproblémy, rekurze a Master theorem
 - Příklady na základní typy výpočetní složitosti (rekurze přijde později)
 - *Bonifikovaný kvíz 2 (bude-li čas): Výpočetní složitost algoritmů*
- **Stromy a jejich vlastnosti** (úvod)
 - Základní charakteristiky, rozdělení a funkce

Typické výpočetní složitosti

Name ↕	Complexity class ↕	Running time ($T(n)$) ↕	Examples of running times ↕	Example algorithms ↕
constant time		$O(1)$	10	Determining if an integer (represented in binary) is even or odd
inverse Ackermann time		$O(\alpha(n))$		Amortized time per operation using a disjoint set
iterated logarithmic time		$O(\log^* n)$		Distributed coloring of cycles
log-logarithmic		$O(\log \log n)$		Amortized time per operation using a bounded priority queue ^[2]
logarithmic time	DLOGTIME	$O(\log n)$	$\log n, \log(n^2)$	Binary search
polylogarithmic time		$\text{poly}(\log n)$	$(\log n)^2$	
fractional power		$O(n^c)$ where $0 < c < 1$	$n^{1/2}, n^{2/3}$	Searching in a kd-tree
linear time		$O(n)$	n	Finding the smallest or largest item in an unsorted array, Kadane's algorithm
"n log star n" time		$O(n \log^* n)$		Seidel's polygon triangulation algorithm.
quasilinear time		$O(n \log n)$	$n \log n, \log n!$	Fastest possible comparison sort; Fast Fourier transform.
quadratic time		$O(n^2)$	n^2	Bubble sort; Insertion sort; Direct convolution
cubic time		$O(n^3)$	n^3	Naive multiplication of two $n \times n$ matrices. Calculating partial correlation.
polynomial time	P	$2^{O(\log n)} = \text{poly}(n)$	$n, n \log n, n^{10}$	Karmarkar's algorithm for linear programming; AKS primality test
quasi-polynomial time	QP	$2^{\text{poly}(\log n)}$	$n^{\log \log n}, n^{\log n}$	Best-known $O(\log^2 n)$ -approximation algorithm for the directed Steiner tree problem.
sub-exponential time (first definition)	SUBEXP	$O(2^{n^\epsilon})$ for all $\epsilon > 0$	$O(2^{\log n^{\log \log n}})$	Assuming complexity theoretic conjectures, BPP is contained in SUBEXP. ^[3]
sub-exponential time (second definition)		$2^{o(n)}$	$2^{n^{1/3}}$	Best-known algorithm for integer factorization and graph isomorphism
exponential time (with linear exponent)	E	$2^{O(n)}$	$1.1^n, 10^n$	Solving the traveling salesman problem using dynamic programming
exponential time	EXPTIME	$2^{\text{poly}(n)}$	$2^n, 2^{n^2}$	Solving matrix chain multiplication via brute-force search
factorial time		$O(n!)$	$n!$	Solving the traveling salesman problem via brute-force search
double exponential time	2-EXPTIME	$2^{2^{\text{poly}(n)}}$	2^{2^n}	Deciding the truth of a given statement in Presburger arithmetic

Zdroj: http://en.wikipedia.org/wiki/Time_complexity

Zjednodušené závěry

- Počet cyklů s kompletními vstupními daty: mocnina rozsahu vstupních dat n :
 - 1 cyklus: $O(n)$
 - 2 vnořené cykly: $O(n^2)$
 - 3 vnořené cykly: $O(n^3)$ -
 - Podobné je to u vícestupňových algoritmů (MergeSort, HeapSort, kde se ale obvykle jedná o rekurzi: $O(n \cdot \log(n))$)
- **Půlí (dělí)-li** se vstupní data uvnitř cyklu nebo rekurze: logaritmická složitost o základu 2
 - $\log_2(n)$ zapisujeme v CS jako $\log(n)$ nebo $\lg(n)$, tj. nejedná se o $\log_{10}(n)$
 - Typicky jde o vyhledávání v uspořádaných binárních stromech
- Naopak **(zdvoj)násobení** vstupních dat (Fibonacci, Hanojské věže) vede na exponenciální složitost

Vnořené cykly v datech
dominují zbytku algoritmu

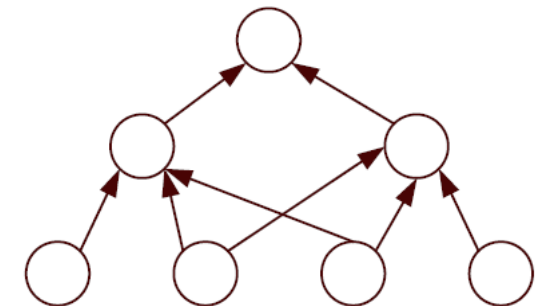
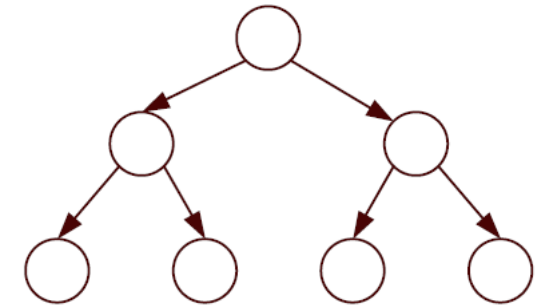
Složitosti nad vybranými obecnými implementačními DS

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
<u>Array</u>	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Stack</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Queue</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Singly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Doubly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$\theta(n)$
<u>Skip List</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n \log(n))$
<u>Hash Table</u>	N/A	$\theta(1)$	$\theta(1)$	$\theta(1)$	N/A	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Binary Search Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>Cartesian Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$
<u>B-Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>Red-Black Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>Splay Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>AVL Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$
<u>KD Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$\theta(n)$

Zdroj: <https://www.bigocheatsheet.com/>

Řešení rozkladem na podproblémy

- Rozděl a panuj (Divide and Conquer)
 - Podproblémy jsou disjunktní
 - Shora-dolů
 - Obvykle rekurzivně
- Dynamické programování
 - Podproblémy se mohou překrývat
 - Zdola nahoru (zapamatování, memoizing) i shora dolů (tabulka)
 - Iterativně i rekurzivně



Faktoriál, rozdíl mezi iterací a rekurzí

```
int fakt_REK(int n) {  
    if (n<=0) return 1;  
    return n* fakt_REK(n-1);  
}
```

```
int fakt_ITER(int n) {  
    int i, f = 1;  
    for (i=2; i<=n; i++) f *= i;  
    return f;  
}
```

Recursive Factorial

5 Factorial

```
def factorial(n):  
    if (n <= 1):  
        return 1  
    else:  
        subSolution = factorial(n - 1)  
        solution = subSolution * n  
        return solution
```

factorial	n	5
	subValue	
	returnValue	
factorial	n	4
	subValue	
	returnValue	
factorial	n	3
	subValue	
	returnValue	
factorial	n	2
	subValue	
	returnValue	
factorial	n	1
	subValue	
	returnValue	

Fibonacci rekurzivně

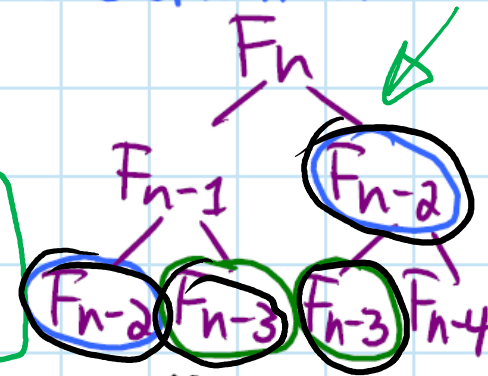
Fibonacci numbers: $F_1 = F_2 = 1$; $F_n = F_{n-1} + F_{n-2}$

– naïve algorithm: follow recursive definition

fib(n):

if $n \leq 2$: return 1

else: return $\text{fib}(n-1) + \text{fib}(n-2)$

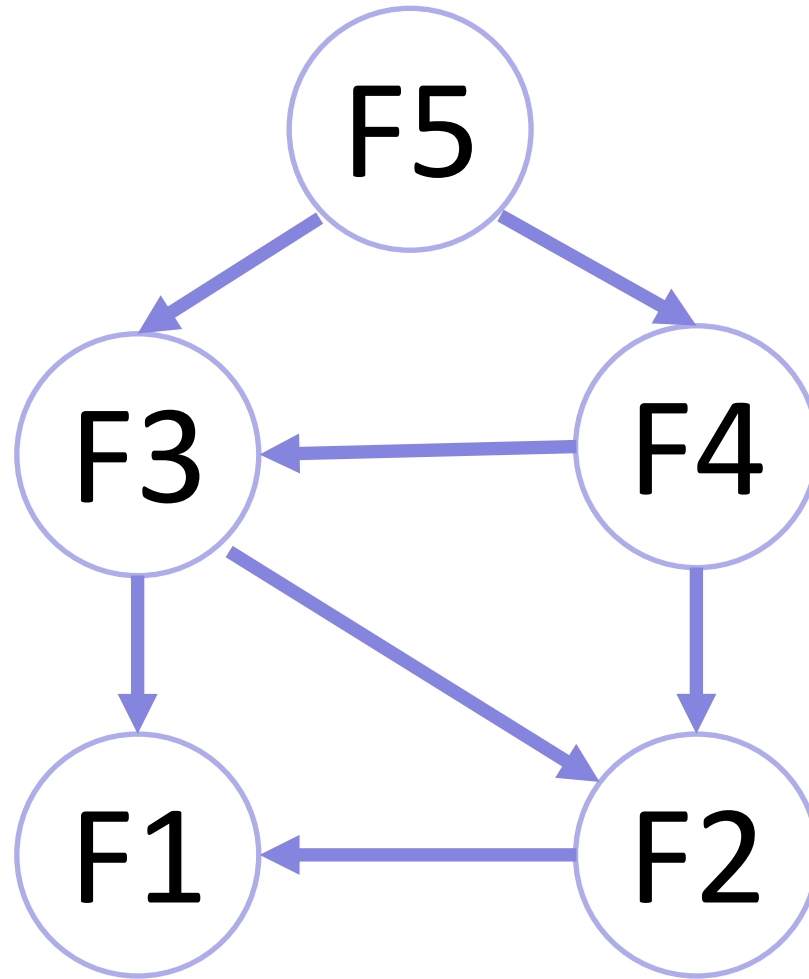


$$\Rightarrow T(n) = T(n-1) + T(n-2) + O(1) \approx \varphi^n$$

$$\geq 2T(n-2) + O(1) \geq 2^{n/2}$$

EXPONENTIAL – BAD!

Fibonacci dynamickým programováním



Nejedná se o strom --> podproblémy se překrývají

Fibonacci se zapamatováním (top-down)

- simple idea: memoize
memo = {}
fib(n):
→ if n in memo: return memo[n]
else: if n ≤ 2: f = 1
else: f = fib(n-1) + fib(n-2)
memo[n] = f
return f
⇒ $T(n) = T(n-1) + O(1) = O(n)$

Zdroje: <http://courses.csail.mit.edu/6.006/spring08/notes/lecture19.pdf>
<https://www.cs.usfca.edu/~galles/visualization/DPFib.html>

Fibonacci, animované srovnání

Dynamic Programming (Fibonacci)

10

Fibonacci Recursive

Fibonacci Table

Fibonacci Memoized

```
def fib(n):  
    if (n <= 1)  
        return 1  
    else  
        return fib(n-1) + fib(n-2)  
  
fib(10)
```



Výpočet složitosti rekurentních relací

(Varianta Divide and conquer)

Mějme konstanty $a \geq 1$, $b > 1$ a funkci $f(n)$ se složitostí $\Theta(n^d)$ pro $d \geq 0$. Pro výpočetní složitost rekurentní funkce:

$$T(n) = aT(n/b) + f(n)$$

pak platí, že:

$$T(n) = \Theta(n^d) \quad \text{když} \quad a < b^d$$

$$T(n) = \Theta(n^d \log n) \quad \text{když} \quad a = b^d$$

$$T(n) = \Theta(n^{\log_b a}) \quad \text{když} \quad a > b^d$$

Master theorem nebo Cookbook

Omezení: Master theorem **neplatí**, když:

$T(n)$ není monotónní, např. $T(n) = \sin(n)$

$f(n)$ není polynomiální, např. $f(n) = 2^n$

a není konstanta, např. $a = 2n$

$a < 1$

Příklad:

Příklad: $T(n) = 8T(n/2) + 1000n^2$

$a=8, b=2, d=2$
 $b^d = 2^2 = 4$
 $a > b^d \Rightarrow 8 > 4$
 $\log_2 8 = 3$
 $\Rightarrow n^3$

$T(n) = T(n/2) + T(n/2) + n^2 = 2T(n/2) + n^2$
 $a=2, b=2, d=1$
 $b^d = 2^1 = 2$
 $a > b^d \Rightarrow 2 > 2$
 $\Rightarrow n \cdot \log n$

Výpočet složitosti rekurentních relací (Aproximate Subtract and conquer)

Mějme konstanty $a \geq 1$, $b > 1$ a funkci $f(n)$ se složitostí $\Theta(n^d)$ pro $d \geq 0$. Pro výpočetní složitost rekurentní funkce:

$$T(n) = aT(n-b) + f(n)$$

pak platí, že:

$$T(n) = \Theta(n^d) \quad \text{když} \quad a < 1$$

$$T(n) = \Theta(n^{d+1}) \quad \text{když} \quad a = 1$$

$$T(n) = \Theta(n^d a^{n/b}) \quad \text{když} \quad a > 1$$

$a=2$
 $b=1$
 $d=0$

$\frac{n}{b}$

$a < 1$
 $a = 1$

$O(1 \cdot 2^n) = 2^n$

Příklad: Hanojské věže

Příklad.

$$T(n) = T(n-1) + 1 + T(n-1)$$
$$= 2T(n-1) + 1$$

```
FUNCTION MoveTower(disk, source, dest, spare)
IF disk == 0, THEN:
    move disk from source to dest
ELSE:
    MoveTower(disk - 1, source, spare, dest)
    move disk from source to dest
    MoveTower(disk - 1, spare, dest, source)
END IF
```

Seřadte následující funkce podle asymptotické složitosti:

n $n \log n$ 63 n^2 2^n n^5 $\log n$

Určete dominantní člen a celkovou $O(f(n))$ těchto funkcí:

$$5 + 0.001n^3 + 0.025n$$

$$100n + 0.01n^2$$

$$500n + 100n^{1.5} + 50n \log_{10} n$$

$$0.01n + 100n^2$$

$$0.3n + 5n^{1.5} + 2.5 \cdot n^{1.75}$$

$$2n + n^{0.5} + 0.5n^{1.25}$$

$$n^2 \log_2 n + n(\log_2 n)^2$$

$$0.01n \log_2 n + n(\log_2 n)^2$$

$$n \log_3 n + n \log_2 n$$

$$100n \log_3 n + n^3 + 100n$$

$$3 \log_8 n + \log_2 \log_2 \log_2 n$$

$$0.003 \log_4 n + \log_2 \log_2 n$$

Příklady, funkce

Řešení

1) Seřadte následující funkce podle asymptotické složitosti:

$$\begin{array}{ccccccc} n & n \log n & 63 & n^2 & 2^n & n^5 & \log n \\ 63 & \log n & n & n \log n & n^2 & n^5 & 2^n \end{array}$$

2) Určete dominantní člen a celkovou $O(f(n))$ těchto funkcí:

$\rightarrow 5 + \underline{0.001n^3} + 0.025n$	$O(n^3)$	$100n + \underline{0.01n^2}$	$O(n^2)$
$\downarrow$ $500n + \underline{100n^{1.5}} + 50n \log_{10} n$	$O(n^{1.5})$	$0.01n + \underline{100n^2}$	$O(n^2)$
$0.3n + 5n^{1.5} + \underline{2.5 \cdot n^{1.75}}$	$O(n^{1.75})$	$2n + n^{0.5} + \underline{0.5n^{1.25}}$	$O(n^{1.25})$
$\underline{n^2 \log_2 n} + n(\log_2 n)^2$	$O(n^2 \log_2 n)$	$0.01n \log_2 n + \underline{n(\log_2 n)^2}$	$O(n(\log_2 n)^2)$
$\textcircled{n \log_3 n} + \underline{n \log_2 n}$	$O(n \log_2 n)$	$100n \log_3 n + \underline{n^3} + 100n$	$O(n^3)$
$\underline{3 \log_8 n} + \log_2 \log_2 \log_2 n$	$O(\log_2 n)$	$\downarrow$ $0.003 \log_4 n + \underline{\log_2 \log_2 n}$	$O(\log_2 n)$

Příklady, srovnání

Funkce *first()* a *second()* aplikujme na seřazené pole $a[i]$

```
int first(int x) {  
    for (int i=0; i<N; i++)  
        if (a[i]==x) return i;  
    return -1;  
}
```

```
int second(int x) {  
    int d=0, h=N-1, k;  
    while (d<=h) {  
        k = (h-d)/2;  
        if (a[k]<x) d = k+1;  
        else if (a[k]>x) h = k-1;  
        else return k;    }  
    return -1; }
```

- Slovně popište jejich funkci.
- Určete $O(f(n))$ u obou.
- Jaký je konkrétní poměr těchto složitostí pro $N = 1000$?

Příklady, srovnání

Řešení

Funkce *first()* a *second()* aplikujme na seřazené pole $a[i]$

```
int first(int x) {  
    for (int i=0; i<N; i++)  
        if (a[i]==x) return i;  
    return -1;  
}
```

$O(N)$
1000

```
int second(int x) {  
    int d=0, h=N-1, k;  
    while (d<=h) {  
        k = (h-d)/2;  
        if (a[k]<x) d = k+1;  
        else if (a[k]>x) h = k-1;  
        else return k;  
    }  
    return -1;  
}
```



$O(\log N)$
 $2^{10} \sim 1024$
 $1000/10 = 100x$

- Slovně popište jejich funkci.
- Určete $O(f(n))$ u obou.
- Jaký je konkrétní poměr těchto složitostí pro $N = 1000$?

Příklady, iterace 1

```
int a = 0, b = 0;
for (i = 0; i < N; i++) {a = a + i};
for (j = 0; j < M; j++) {b = b + j};
```

```
int a = 0;
for (i = 0; i < N; i++) {
    for (j = N; j > i; j--) {
        a = a + i + j;
    }
}
```

Příklady, iterace 1

Řešení

```
int a = 0, b = 0;  
for (i = 0; i < N; i++) {a = a + i};  
for (j = 0; j < M; j++) {b = b + j};
```

N
 M
 $O(N+M)$

```
int a = 0;  
for (i = 0; i < N; i++) {  
    for (j = N; j > i; j--) {  
        a = a + i + j;  
    }  
}
```

$O(N^2)$
 $\frac{N(N+1)}{2}$
 $O\left(\frac{N(N+1)}{2}\right)$

Příklady, iterace 2

```
int i, j, a = 0;
for (i = n / 2; i <= n; i++) {
    for (j = 2; j <= n; j = j * 2) {
        a = a + n / 2;
    }
}
```

```
int a = 0, i = N;
while (i > 0) {
    a = a + i;
    i = i / 2;
}
```

Příklady, iterace 2

Řešení

```
int i, j, a = 0;
for (i = n / 2; i <= n; i++) {
    for (j = 2; j <= n; j = j * 2) {
        a = a + n / 2;
    }
}
```

$N=16$ 2 4 8 16

```
int a = 0, i = N;
while (i > 0) {
    a = a + i;
    i = i / 2;
}
```

16
8
4
2

$\rightarrow O(\log n)$

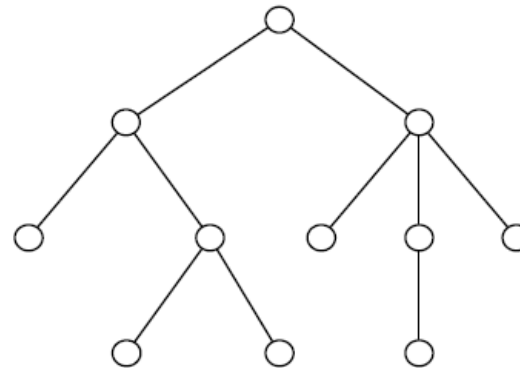
$n/2$

$\log n$

$O\left(\frac{n}{2} \cdot \log n\right)$
 $= O(n \log n)$

Stromy

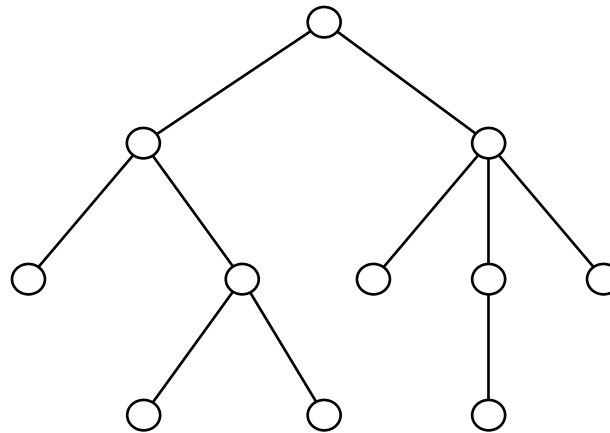
- Podle teorie grafů:
 - **Strom** je souvislý neorientovaný graf bez kružnic
 - **Kořenový strom** je orientovaný strom, ve kterém jeden uzel – kořen – je spojen s každým jiným uzlem jedinou orientovanou cestou
(Souvislý orientovaný graf, který neobsahuje cykly.)
- V informatice se obvykle rozumí: strom = kořenový strom
- Příklad grafického vyjádření:



- Orientaci hran obvykle nekreslíme ale předpokládáme shora dolů

Stromy

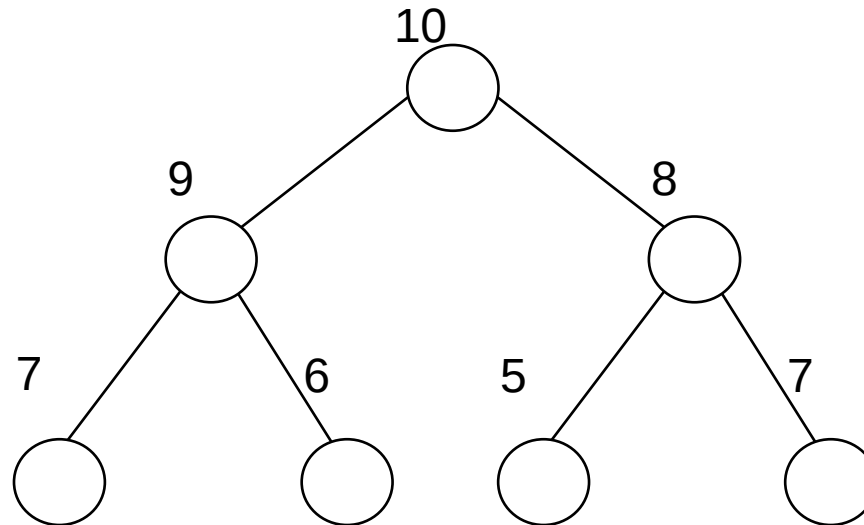
- Podle teorie grafů:
 - **Strom** je souvislý neorientovaný graf bez kružnic
 - **Kořenový strom** je orientovaný strom, ve kterém jeden uzel – kořen – je spojen s každým jiným uzlem jedinou orientovanou cestou
(Souvislý orientovaný graf, který neobsahuje cykly.)
- V informatice se obvykle rozumí: strom = kořenový strom
- Příklad grafického vyjádření:



- Orientaci hran obvykle nekreslíme ale předpokládáme shora dolů

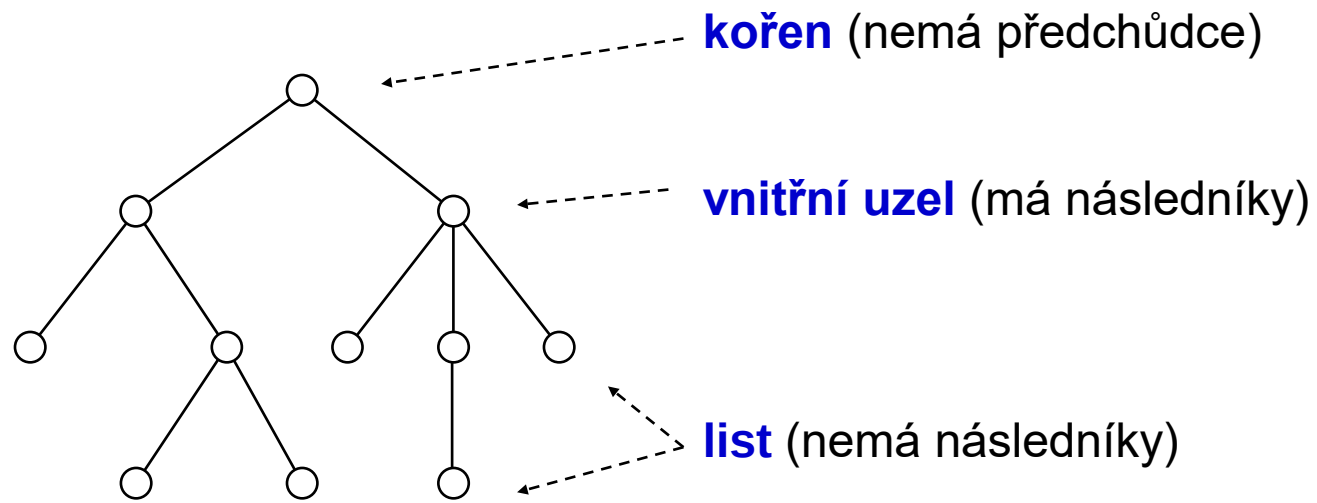
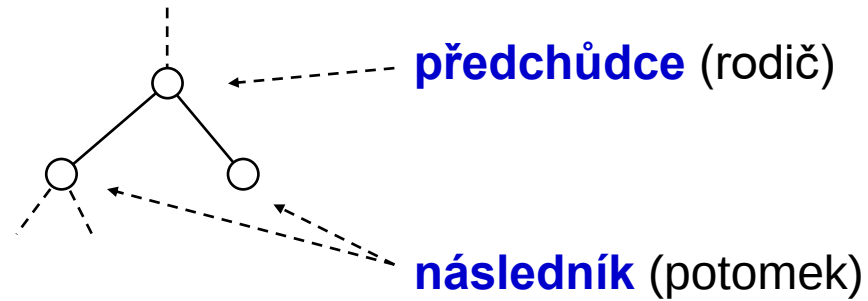
Stromy

- Strom bývá obvykle **uzlově ohodnocený**
 - každý uzel obsahuje hodnotu určitého typu
 - hodnota uzlu (případně její část, jde-li o strukturovanou hodnotu) je **klíčem uzlu**
- Příklad:



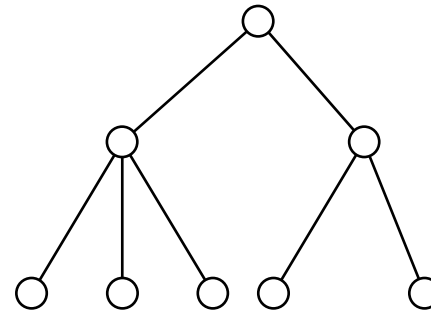
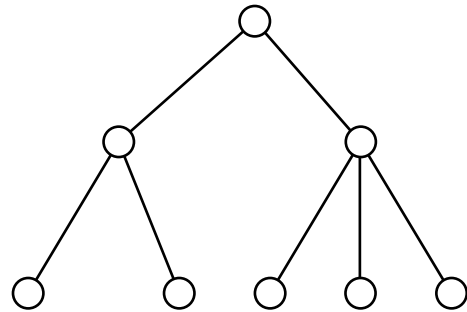
Stromy

- Některé pojmy



Stromy

- Stromy chápeme jako **uspořádané**, tzn. následníci každého vnitřního uzlu jsou pevným způsobem uspořádáni (první, druhý, ..., v grafickém vyjádření zleva doprava)
- Příklad: následující dva stromy jsou různé



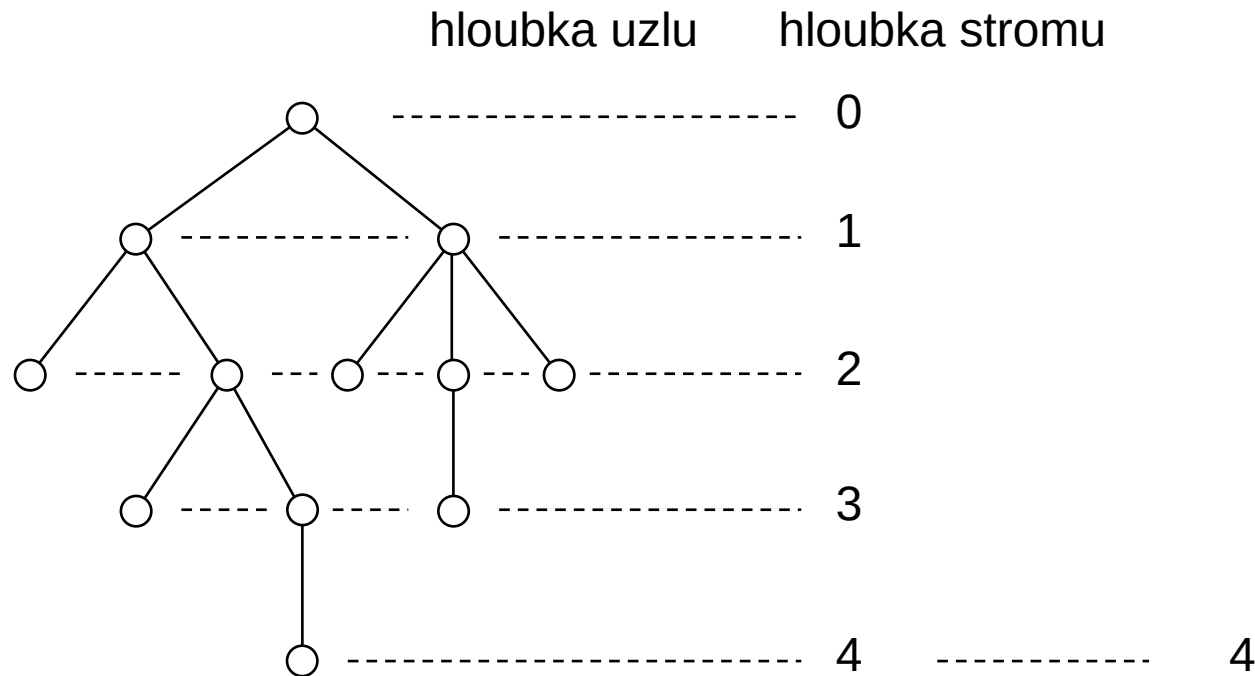
Stromy

- **Hloubka uzlu**

- délka (počet hran) cesty z kořene do uzlu

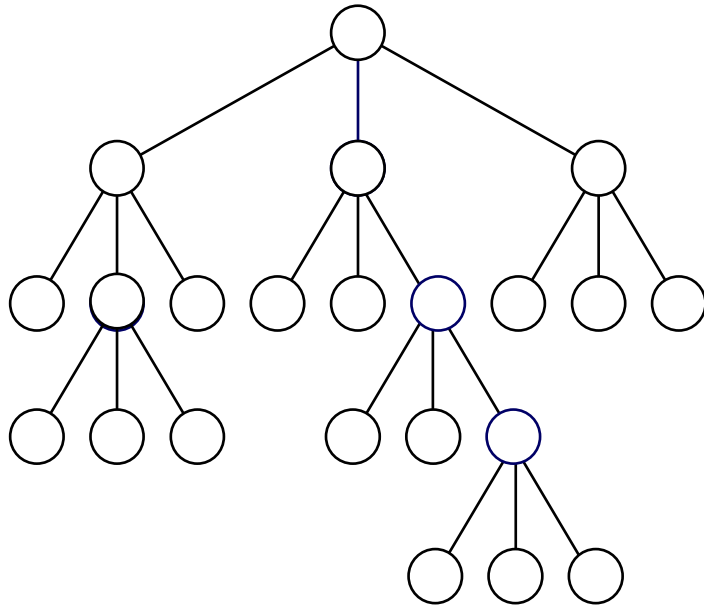
- **Hloubka (výška) stromu**

- délka nejdelší cesty

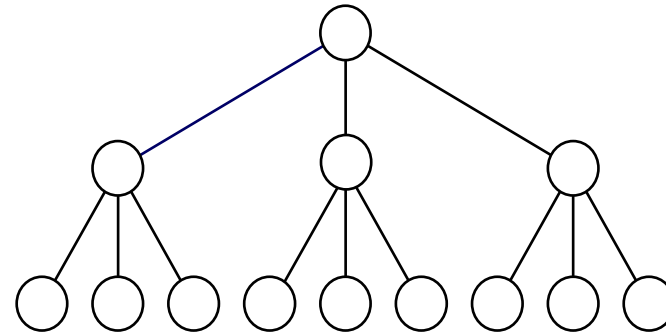


Stromy

- **Pravidelný strom** stupně r ($r > 0$):
 - každý uzel má 0 nebo r následníků
- **Úplný pravidelný strom** stupně r ($r > 0$):
 - všechny listy mají hloubku stromu



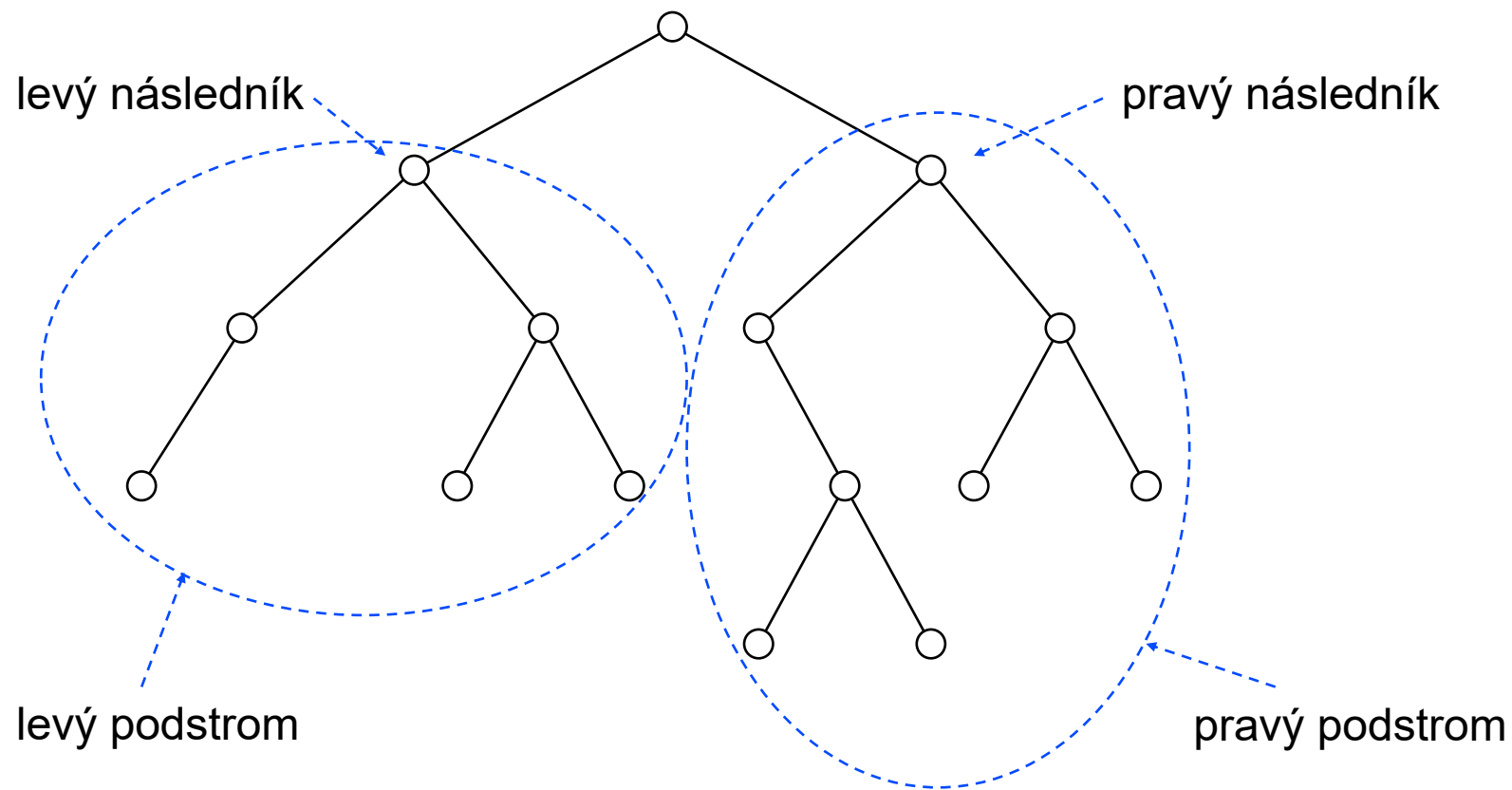
pravidelný strom stupně 3



úplný pravidelný strom stupně 3

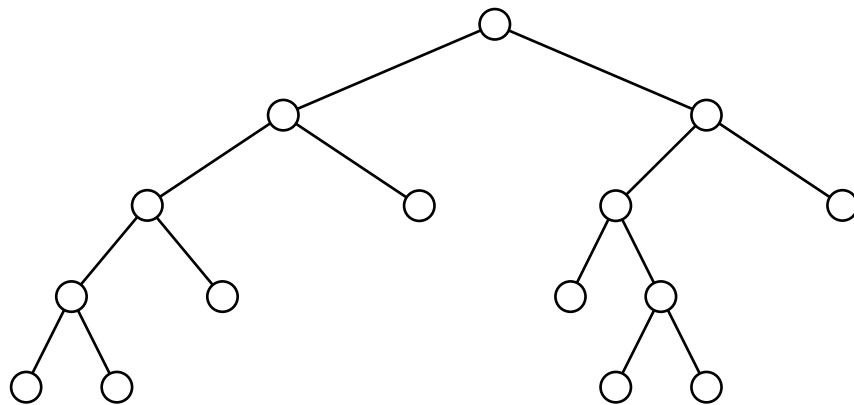
Binární stromy

- **Binární strom:** každý uzel má nanejvýš dva následníky

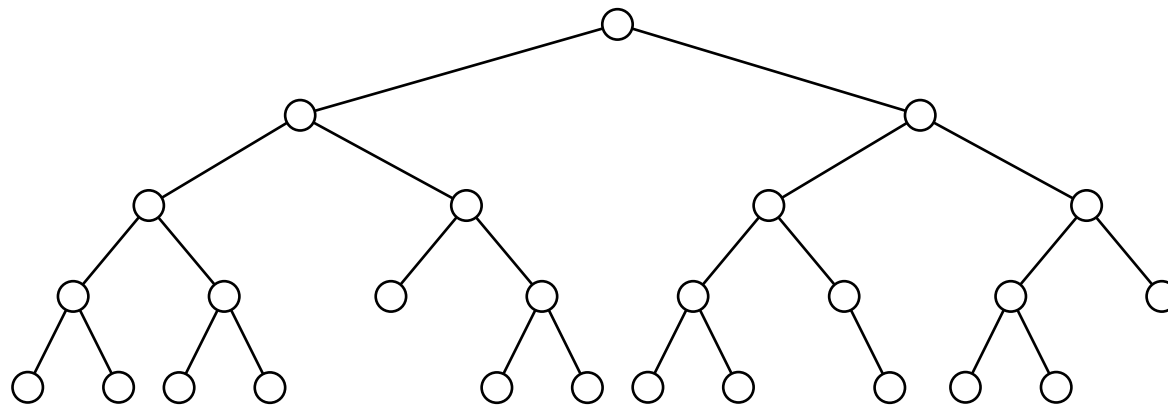


Binární stromy

- **Pravidelný binární strom:** každý uzel má 0 nebo 2 následníků

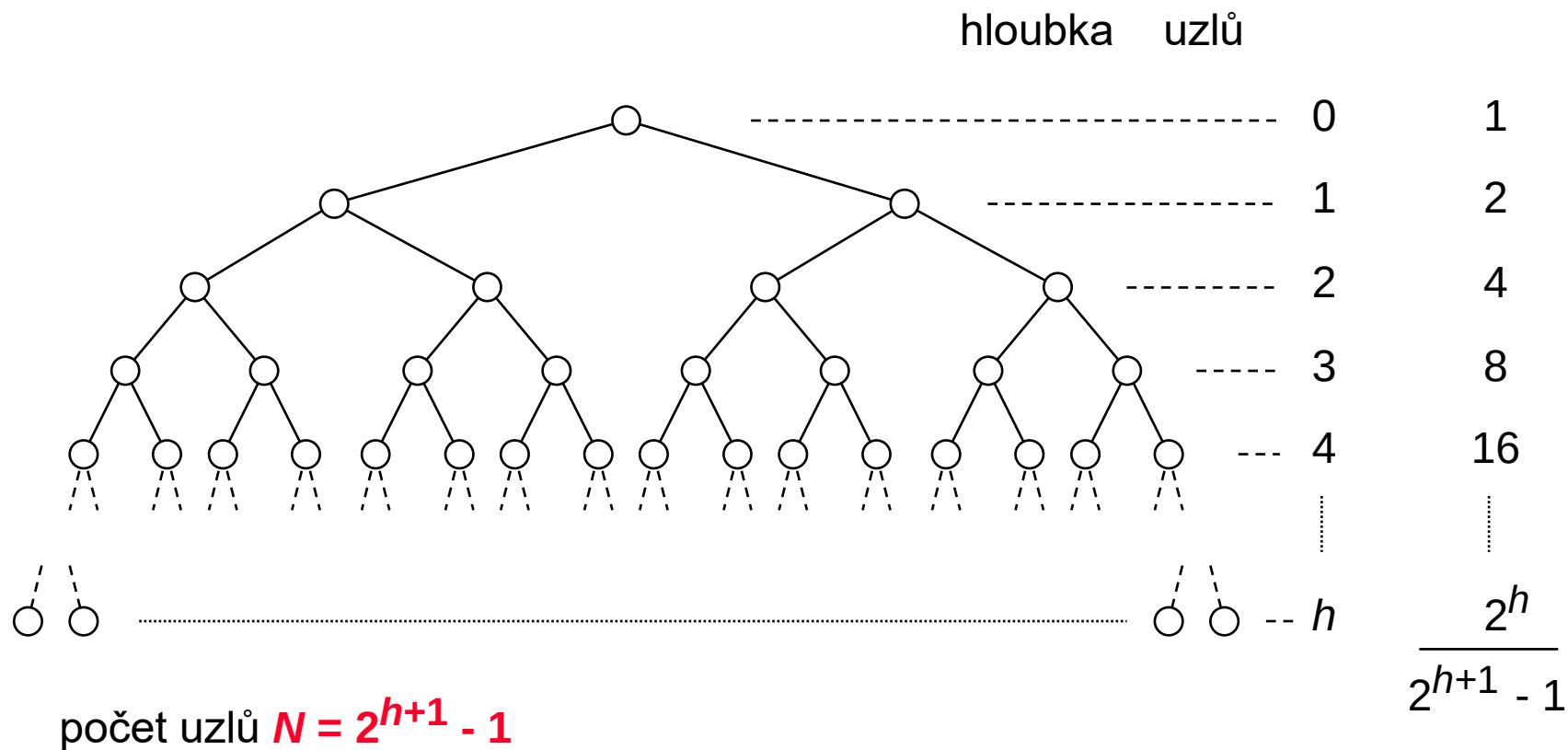


- **Výškově vyvážený binární strom:** hloubky všech listů se liší nanejvýš o 1



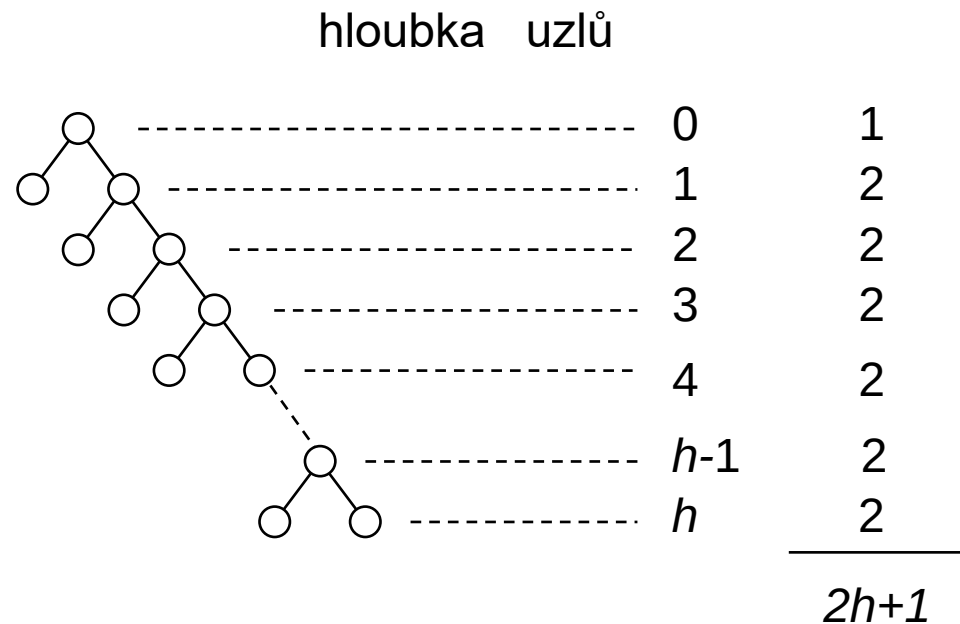
Hloubka stromu a počet uzlů

- Úplný pravidelný binární strom (je vyvážený)



Hloubka stromu a počet uzlů

- Extrémně nevyvážený pravidelný binární strom

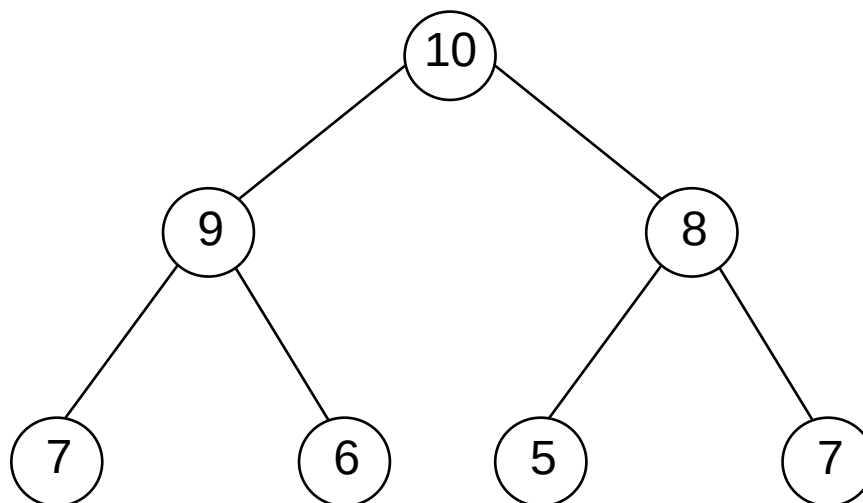


počet uzlů $N = 2h + 1$

hloubka $h = (N - 1) / 2$

Haldově uspořádaný strom

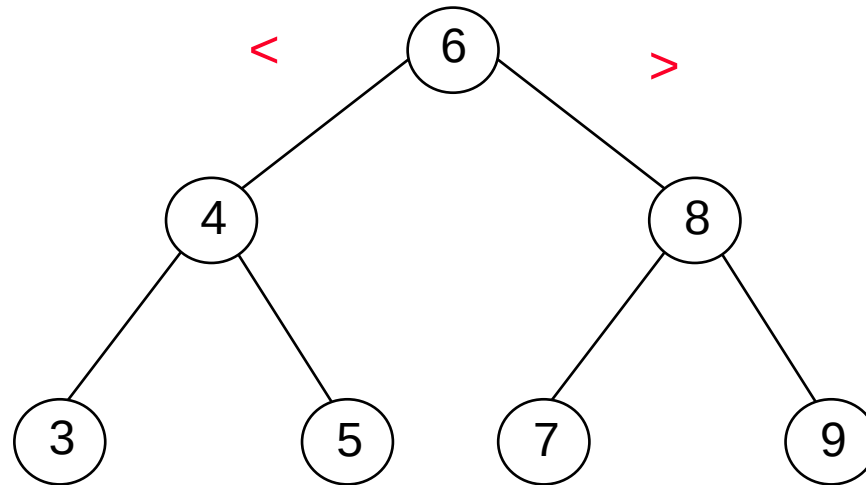
- **Haldově uspořádaný strom** je binární strom, ve kterém klíč každého uzlu je větší nebo roven klíčům následníků (**max-heap**)



- Alternativní definice: Haldově uspořádaný strom je binární strom, ve kterém klíč každého uzlu je menší nebo roven klíčům následníků (**min-heap**)

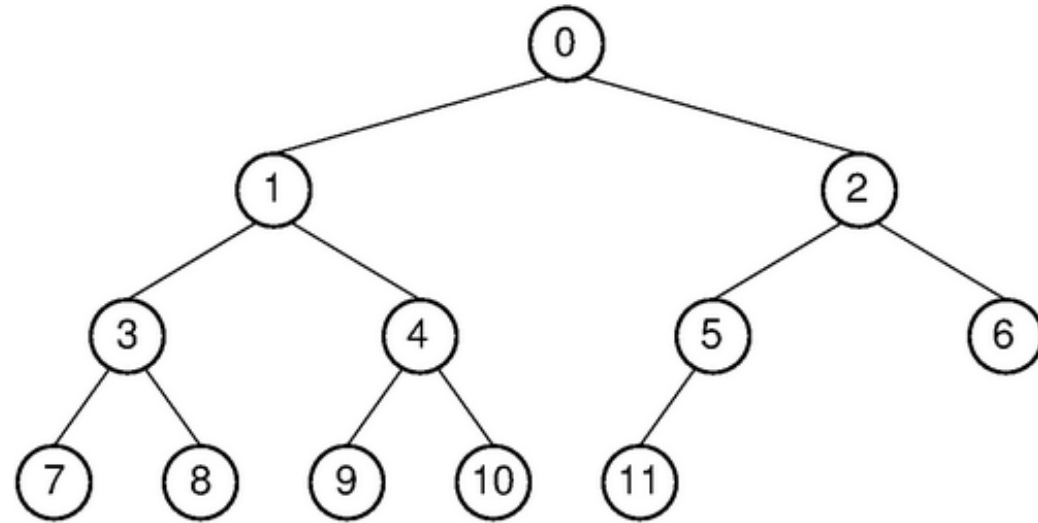
Vyhledávací strom

- **Vyhledávací strom** je binární strom, ve kterém klíč každého vnitřního uzlu je větší než klíče uzlů v levém podstromu a menší než klíče uzlů v pravém podstromu



- Alternativní definice: Vyhledávací strom je binární strom, ve kterém klíč každého vnitřního uzlu je menší než klíče uzlů v levém podstromu a větší než klíče uzlů v pravém podstromu

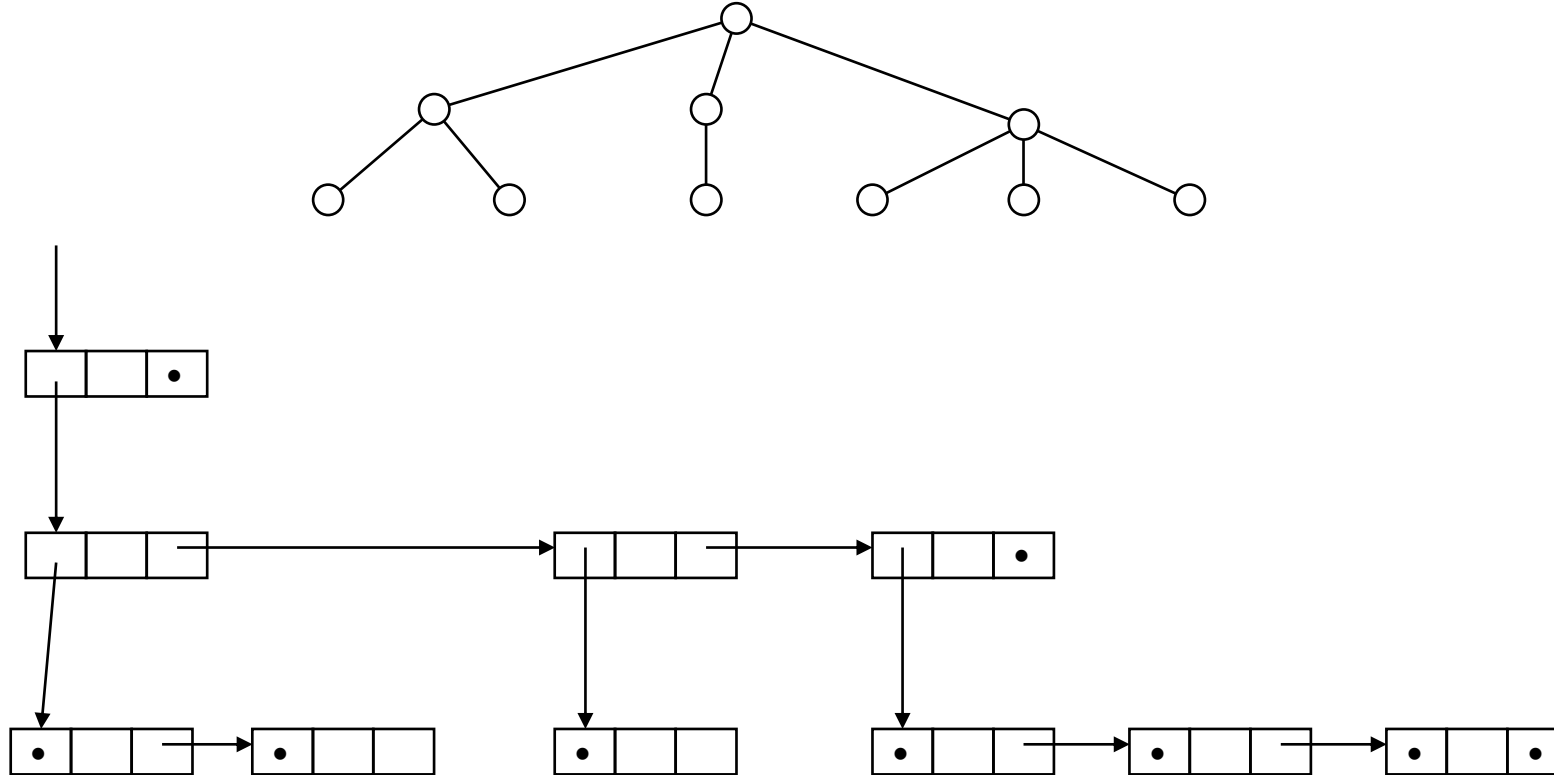
Implementace binárního stromu 2D polem



UZEL	0	1	2	3	4	...
Rodič	R	0	0	1	1	
Levý sourozenec	---	---	1	---	3	
Pravý sourozenec	---	2	---	4	---	
Levý potomek	1	3	5	7	9	
Pravý potomek	2	4	6	8	10	

Programová realizace stromu

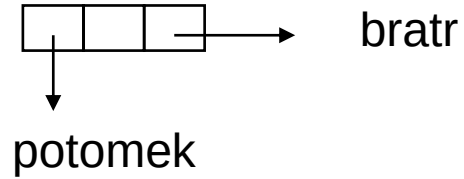
- Kořenový strom, jehož uzly mohou mít libovolný počet následníků, lze realizovat **pomocí spojového seznamu**, jehož prvky jsou zase spojové seznamy



- Nelineární spojová struktura

Programová realizace stromu

- Pro uzly zavedeme třídu



```
class Node {  
public:  
    Node *child;  
    T      info;  
    Node *brother;  
    /* konstruktor uzlu */  
    Node(Node *c, T x, Node *b) {  
        child =c; info = x; brother = b;  
    }  
};
```

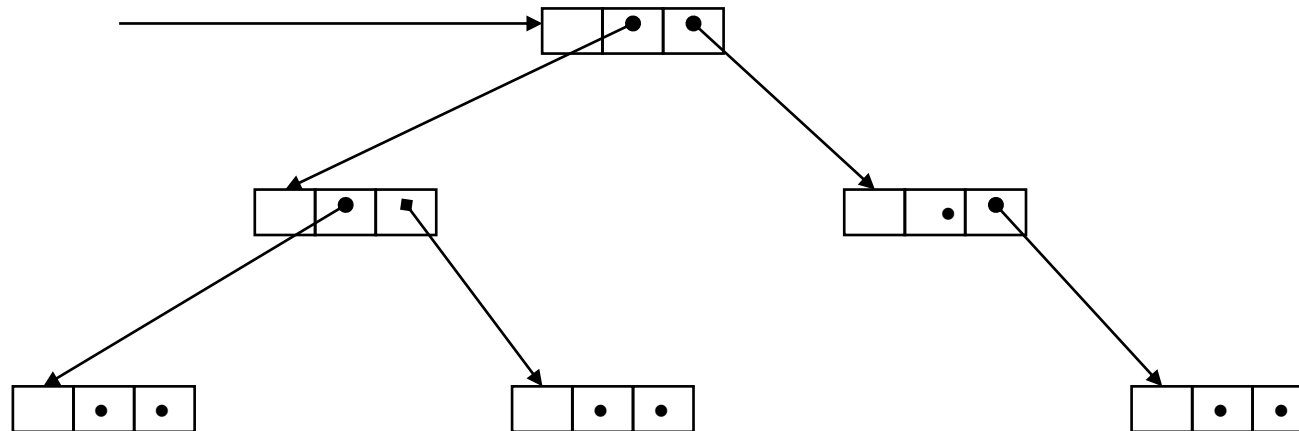
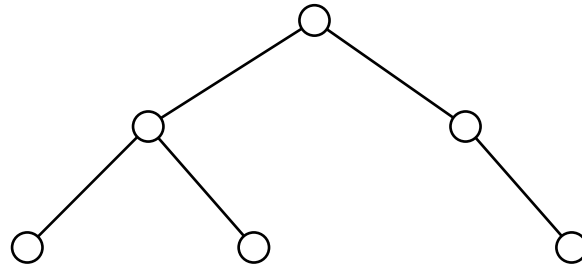
Programová realizace stromu

- Jestliže strom je pravidelný **stupně r** nebo každý uzel má nanejvýš r následníků, můžeme spojový seznam nahradit objektem, který obsahuje **pole ukazatelů na následníky**
- Například:

```
class Node {  
    enum {R = 6;};  
    T info;  
    Node* childs[R];          /* pole ukazatelů*/  
    // konstruktor uzlu  
    Node(T x) {  
        info = x;  
        for (int i=0; i<R; i++)  
            childs[i] = NULL;  
    }  
};
```

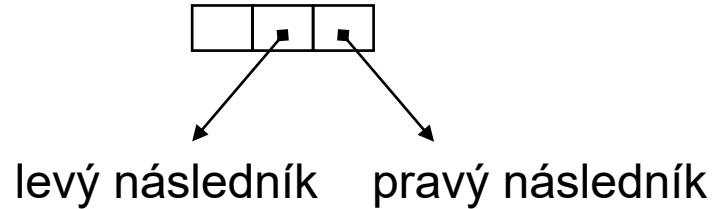
Programová realizace binárního stromu

- Binární strom lze realizovat pomocí nelineární spojové struktury, jejíž uzly budou obsahovat ukazatele na levého a pravého následníka



Programová realizace binárního stromu

- Pro uzly zavedeme třídu



```
class Node {  
public:  
    T    info;  
    Node *left;  
    Node *right;  
    // konstruktor uzlu  
    Node(T i, Node l=NULL; Node r=NULL) {  
        info = i; left = l; right = r;  
    }  
};
```


Programová realizace binárního stromu

- Příklad: funkce pro vytvoření náhodného binárního stromu dané maximální hloubky

```
inline int random(int m) {  
    // výsledkem je pseudonahodne cislo z intervalu 0 az m-1  
    return rand()%m;  
}  
  
Node *randTree(int depth) {  
    if (depth<=0 || random(10)>7) return NULL;  
    return new Node(random(100), randTree(depth-1),  
                    randTree(depth-1));  
}
```

pokud tento test vypustíme, bude vytvořený strom vždy pravidelný a úplný

Binární strom, zadání

(komentované řešení viz MP4 animace)

Mějme binární vyhledávací strom se strukturou uzlu:

```
struct Node { int data;  Node* left;  Node* right; };
```

a funkci pro vytvoření uzlu :

```
Node* GetNode(int data) {  
    Node* newNode = new Node();  
    newNode->data = data;  
    newNode->left = NULL;  
    newNode->right = NULL;  
    return newNode; }
```

Jak by mohly vypadat funkce pro:

- a) Vložení uzlu se zadanou hodnotou klíče?
- b) Nalezení uzlu se zadanou hodnotou klíče?

Binární strom, řešení

```
Node* Insert(Node* root, int data) {  
    if (root == NULL) root = GetNode(data);  
    else if (data < root->data) {  
        root->left = Insert(root->left, data);  
    }  
    else {  
        root->right = Insert(root->right, data);  
    }  
    return root; }
```

```
bool Search(Node* root, int data) {  
    if (root == NULL) return false;  
    else if (root->data == data) return true;  
    else if (data < root->data) return Search(root->left, data);  
    else return Search(root->right, data); }
```

Binární strom, paměť

```
int main() {  
    Node* root = NULL;  
    root = Insert(root, 10);  
    root = Insert(root, 5);  
    root = Insert(root, 20);  
    int number = 30;  
    Search (root, number) ?  
        cout<<"OK":cout << "NO";  
    return 0;  
}
```

Výpis binárního stromu

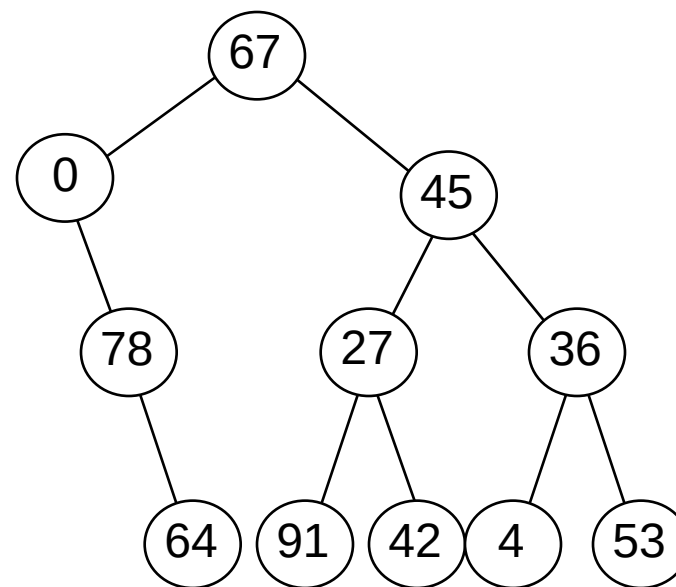
- Předchozí program vytvořil náhodný binární strom (pomocí rekurzivní funkce) , ale jak ho vypsát?
- Opět pomocí rekurzivní funkce, která:
 - vypíše hodnotu v uzlu, na který ukazuje parametr funkce,
 - rekurzivním voláním vypíše uzly v levém podstromu
 - rekurzivním voláním vypíše uzly v pravém podstromu
- Parametrem funkce bude kromě ukazatele na uzel také číslo udávající, kolik mezer má být vypsáno před hodnotou uzlu (odsazení podle hloubky uzlu)

```
void print(Node *p, int s) {  
    cout << setw(s) << ' ';  
    if (p) {  
        cout << p->info << endl;  
        print(p->left, s+1);    /*výpis levého podstromu*/  
        print(p->right, s+1);   /*výpis pravého podstromu*/  
    }  
    else  
        cout << "null" << endl;  
}
```

Výpis binárního stromu

- Program binstrom2.cpp např. vypíše:

```
67
0
  null
78
  null
64
  null
  null
45
27
  91
    null
    null
42
  null
  null
36
  4
    null
    null
53
  null
  null
```



grafické vyjádření vytvořeného stromu

Průchody binárním stromem

- Funkce *print* v programu *binstrom2.cpp* je příkladem průchodu binárním stromem shora dolů v pořadí ***Preorder***
- Další způsoby průchodu binárním stromem jsou *Inorder* a *Postorder*
- **Preorder:** pro každý uzel se provede:
 - operace s hodnotou uzlu
 - vyvolání průchodu pro levého následníka
 - vyvolání průchodu pro pravého následníka
- **Inorder:** pro každý uzel se provede:
 - vyvolání průchodu pro levého následníka
 - operace s hodnotou uzlu
 - vyvolání průchodu pro pravého následníka
- **Postorder:** pro každý uzel se provede:
 - vyvolání průchodu pro levého následníka
 - vyvolání průchodu pro pravého následníka
 - operace s hodnotou uzlu

Výpis binárního stromu

- Průchodem **Inorder**:

```
void printInorder(Node *p, int s) {  
    if (p) {  
        printInorder(p->left, s+1);  
        cout << setw(s) << ' ' << p->info << endl;  
        printInorder(p->right, s+1);  
    }  
    else  
        cout << setw(s) << ' ' << "null" << endl;  
}
```

- Průchodem **Postorder**:

```
void printPostorder(Node *p, int s) {  
    if (p) {  
        printPostorder(p->left, s+1);  
        printPostorder(p->right, s+1);  
        cout << setw(s) << ' ' << p->info << endl;  
    }  
    else  
        cout << setw(s) << ' ' << "null" << endl;  
}
```


Průchod stromem

- Do hloubky (BFS, Breadth-First Search)

- Preorder

- 8, 5, 9, 7, 1, 12, 2, 4, 11, 3

- Inorder

- 9, 5, 1, 7, 2, 12, 8, 4, 3, 11

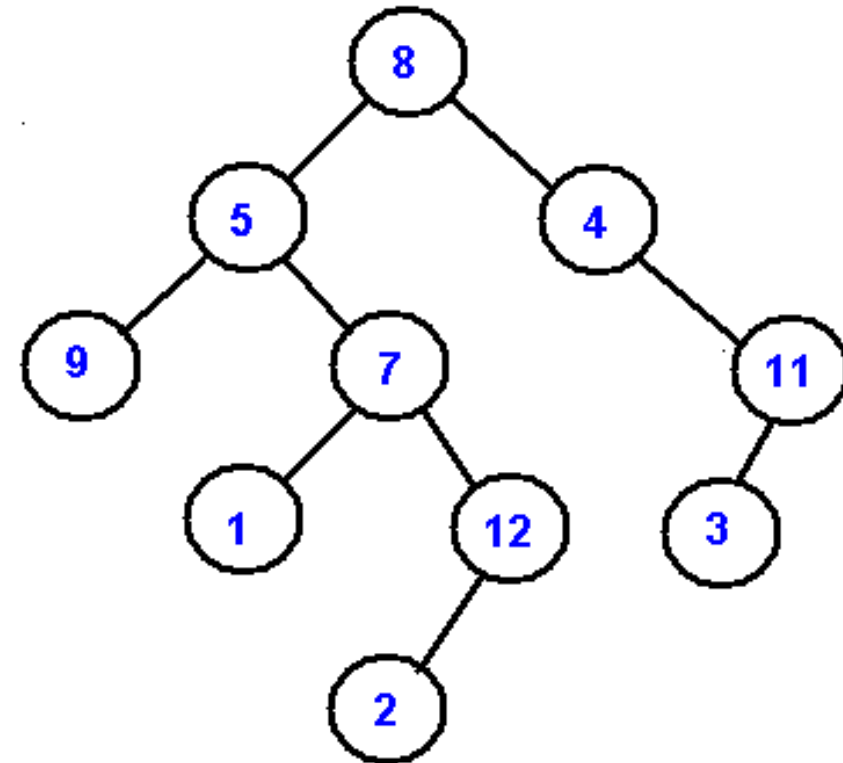
- Postorder

- 9, 1, 2, 12, 7, 5, 3, 11, 4, 8

- Do šířky (DFS, Depth-First Search)

- Level order

- 8, 5, 4, 9, 7, 11, 1, 12, 3, 2

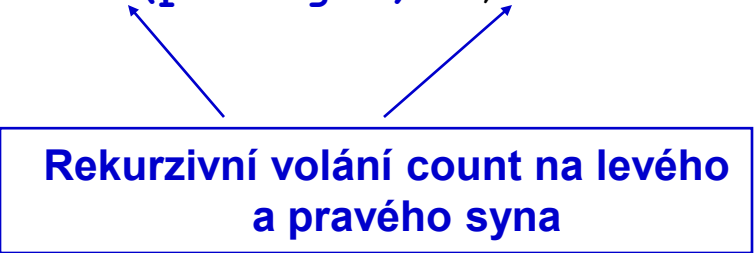


Počet uzlů binárního stromu

- Funkce pro výpočet počtu uzlů binárního stromu

```
int count(Node *p) {  
    if (p)  
        return count(p->left) + count(p->right) + 1;  
    else  
        return 0;  
}
```

Rekurzivní volání count na levého
a pravého syna

A blue rectangular box containing the text 'Rekurzivní volání count na levého a pravého syna'. Two blue arrows originate from the box: one points to the 'count(p->left)' expression in the code, and the other points to the 'count(p->right)' expression.

Hloubka binárního stromu

```
#define max(x,y) ((x)>(y)?(x):(y))

int depth(Node *p) {
    if (p)
        // vrací větší z hloubek levého a pravého podstromu      zvětšenou o
        // kořen stromu
        return max(depth(p->left), depth(p->right)) + 1;
    else
        return -1;
}
```

Implementace **rekurze pomocí zásobníku**

- Funkci pro výpočet počtu uzlů binárního stromu napíšeme **nerekurzivě**
- Postup: použijeme **zásobník ukazatelů** na uzly stromu

```
int count2(Node *p) {  
    Stack<Node*> stc;  
    int cnt = 0;  
    stc.push(p);          /*ukazatel na kořen do zásobníku*/  
    while (!stc.empty()) { /*dokud není zásobník prázdný*/  
        Node *q = stc.pop(); /*odeber z vrcholu zásobníku*/  
        if (q) {  
            cnt++;          /*zvýšení počtu uzlů*/  
  
            /*ukazatel na levého a pravého syna do zásobníku*/  
            stc.push(q->right);  
            stc.push(q->left);    }  
        }  
    }  
    return cnt;  
}
```

Implementace rekurze pomocí zásobníku

- Proceduru pro výpis stromu metodou preorder napíšeme nerekurzivně
- Postup:
 - do zásobníku budeme ukládat dvojice <ukazatel na uzel, počet mezer>
 - opakovaně, dokud se zásobník nevyprázdní, odebereme z vrcholu dvojici, vypíšeme počet mezer, a pokud ukazatel není NULL, vložíme do zásobníku ukazatel na pravého následníka a pak ukazatel na levého následníka (pořadí je důležité); pokud je ukazatel *null*, vypíšeme “null”
 - na začátku vložíme do zásobníku dvojici obsahující ukazatel na kořen a 1

Implementace rekurze pomocí zásobníku

```
struct Pair { /*dvojice <ukazatel na uzel, počet mezer>*/
    Node *ptr;
    int spc;
    Pair(Node *p, int s) {
        ptr = p; spc = s;
    }
};

void print2(Node *p, int s) {
    Stack<Pair> stc;          /*vytvoření zásobníku*/
    stc.push(Pair(p,s));     /*ukazatel na kořen do zásobníku*/
    while (!stc.empty()) {   /*dokud není zásobník prázdný*/
        Pair pair = stc.pop(); /*odeber z vrcholu zásobníku*/
        cout << setw(pair.spc) << ' ';
        if (pair.ptr) {
            cout << pair.ptr->info << endl;
            /*ukazatel na levého a pravého syna a počet mezer do      zásobníku*/
            stc.push(Pair(pair.ptr->right, pair.spc+1));
            stc.push(Pair(pair.ptr->left, pair.spc+1));
        }
        else
            cout << "null" << endl;
    }
}
```